Trabajo 2. Optimización de consultas

Autor: Lucas Hidalgo Herrera

Grado: Doble Grado en Ingeniería Informática y Matemáticas

Asignatura: Administración de Bases de Datos

Profesor: Ignacio José Blanco Medina

Fecha: 28 de abril de 2026

Índice General

1 Introducción **1**

2 Plantear la consulta **1**
2.1 Tablas a relacionar 1
2.2 Consulta 1

3 TDA PlanLogico **2**
3.1 OperacionAlgebraica 3
3.2 TDA NodeLogico 3
3.2.1 Estructura 3
3.2.2 Operaciones privadas 4
3.2.3 Operaciones públicas 5
3.3 TDA PlanLogico 9
3.3.1 Estructura 9
3.3.2 Operaciones privadas 10
3.3.3 Operaciones públicas 10

4 TDA PlanFisico **12**
4.1 TDA NodeFisico 12
4.1.1 Estructura 12
4.1.2 Operaciones privadas 13
4.1.3 Operaciones públicas 13
4.2 TDA PlanFisico 14
4.2.1 Estructura 14
4.2.2 Operaciones privadas 14
4.2.3 Operaciones públicas 14

5 Heurística proyeciones **15**

6 Heurística del producto cartesiano **16**

1 Introducción

A lo largo de este documento se va a realizar el Trabajo 2 de la asignatura de *Administración de Bases de Datos*. Dicho trabajo consiste en documentar, diseñar y especificar los tipos de datos abstractos necesarios para gestionar la optimización de consultas en un Sistema Gestor de Bases de Datos. Dichos tipos de datos abstractos son *TDA NodeLogico*, *TDA PlanLogico*, *TDA NodeFisico* y *TDA PlanFisico*. Además, se explicará la heurística de optimización de proyecciones y la heurística de optimización de productos cartesianos.

2 Plantear la consulta

Para poder realizar el trabajo debemos plantear una consulta en *SQL* con la que trabajar; nuestro objetivo será aplicar nuestras heurísticas de optimización para conseguir la más eficiente de forma estimada.

2.1 Tablas a relacionar

Para poder trabajar la consulta, debemos saber qué tablas relaciona; concretamente supondremos que tendremos **mecánicos** y **reparaciones**. Las tablas aparencen en las figuras 1 y 2.

id_mecanico	nombre	cargo	salario
NUMBER	VARCHAR(20)	VARCHAR(10)	DECIMAL(5,3)

Cuadro 1: Esquema de la relación Mecánico

id_mecanico	matricula	fecha	num_horas
NUMBER	VARCHAR(7)	DATE	NUMBER

Cuadro 2: Esquema de la relación Repara

Donde se cumplen las siguientes restricciones:

- **Mecanico.id_mecanico** es clave primaria.
- **Repara.id_mecanico** es clave externa de Mecanico.id_mecanico.
- **Matricula** y **fecha** son claves externas a otras relaciones. [1]

2.2 Consulta

La consulta sobre la que se basará todo el desarrollo de esta práctica se encuentra en el *listing* 2.2.

[1] Esto no influye pero en el problema de Fundamentos de Bases de Datos del que he tomado la consulta es así.

```sql
SELECT matricula, cargo
FROM (SELECT *
      FROM Repara,Mecanico WHERE)
AS resultado
WHERE Repara.id_mecanico == Mecanico.id_mecanico AND
   num_horas>2;
```

Esta consulta busca conseguir las parejas de coche y cargo de manera que el mecánico con ese cargo ha realizado una reparación de más de dos horas a ese coche. Además, tal y como se pide en el guión del trabajo, se debe proporcionar la representación en árbol de expresión algebraica que se encuentra en la figura 1.

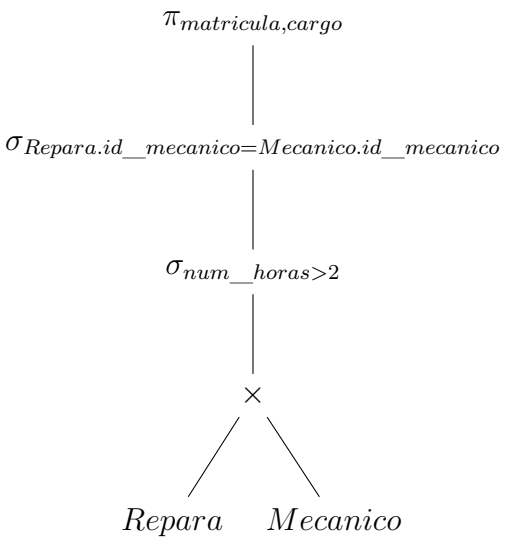

Figura 1: Árbol de expresión algebraica

3 TDA PlanLogico

Si pensamos en lo que vimos en teoría, un plan lógico es un árbol heterogéneo donde sus nodos se dividen en do tipos:

- **Interior**. Todos ellos son operaciones del algebra relacional cuyas operaciones aplicadas a los nodos hoja de forma ascendente dan lugar al resultado de la consulta.
- **Hoja**. Contienen las tablas sobre las que se aplica la consulta, en el pseudocódigo se representarán como *TABLA*.

Por tanto, debemos diseñar un tipo de dato abstracto que represente un árbol heterogéneo cuyos nodos sean de esa forma; por tanto, deberemos crear el tipo de dato abstracto *TDA NodeLogico*.

3.1 OperacionAlgebraica

Entonces, una vez que vemos que debemos crear el tipo de dato abstracto *TDA NodeLógico* debemos saber cómo representar el contenido del nodo; para ello, he decidido crear esta estructura de datos que contiene todas las posibilidades de un nodo de nuestro árbol heterogéneo.

```
    enum TipoOperador = {TABLA, SELECCION, PROYECCION, NATURAL JOIN
   , JOIN, PRODCART};

    struct OperacionAlgebraica{
        TipoOperador operacion;
        vector<cadenaCaracteres> esquemaSalida;
        vector<vector<cadenaCaracteres>> esquemasEntrada;


        //Atributos segun operacion
        cadenaCaracteres nombreTabla; //Solo para TABLA
        cadenaCaracteres predicado; //Solo para SELECCION
        lista<cadenaCaracteres> atributosProyectados; //Solo para
   PROYECCION
        cadenaCaracteres conditionJoin; //Solo para NATURAL JOIN
    }
```

Listing 1: Estructura de una operacion de algebra relacional

Como vemos, hay comentarios sobre la segunda tanda de atributos. Estos comentarios representan que los atributos que aparecen son únicos dependiendo de TipoOperador, de manera que el correcto uso de esta estructura depende del programador.

3.2 TDA NodeLogico

Pasamos a crear el tipo de dato abstracto posiblemente más importante del trabajo; será el corazón del tipo de dato abstracto *TDA PlanLogico*. Tal y como dijimos anteriormente, su contenido será *OperacionAlgebraica*; no obstante, necesitamos poder poner el nodo dentro del árbol por lo que hemos añadido una lista de punteros a hijos (permitiendo que se n-ario)[2] y un puntero a su padre[3].

3.2.1 Estructura

```
class NodeLogico{
    private:
        pointer<NodeLogico> father;
        vector<pointer<NodeLogico>> children;
        OperacionAlgebraica operation;

    /*Métodos*/
}
```

Listing 2: Estructura *NodeLogico*

[2]Realmente solo podrá ser binario a lo sumo.

[3]El puntero al padre será útil en el algoritmo de optimización de productos cartesianos.

3.2.2 Operaciones privadas

```
private:
/*
* Construye el atributo operacion del nodo
*
* @param operacion: operación que alberga el nodo.
* @param esquemaEntrada: según la operación que sea se toma de una manera u otra.
*       TABLA, SELECCION, PROYECCION: solo se toma el primer esquema.
*       NATURAL JOIN, JOIN, PRODCART: se toman los dos primeros
* @param info: tiene una interpretacion u otra segun el valor de operacion:
*       TABLA: toma la primera cadena de caracteres como nombre de la tabla.
*       SELECCION: toma la primera cadena de caracteres como predicado de la seleccion.
*       PROYECCION: toma la lista entera como lista de atributos sobre los que proyectar.
*       NATURAL JOIN: toma la primera cadena de caracteres como condicion de reunion.
*       PRODCART: no lo tiene en cuenta.
*
* @ret operacion que se puede crear con los argumentos pasados.
*/
OperacionAlgebraica buildOperation(TipoOperador operacion, pointer<vector<vector<cadenaCaracteres>>> esquemasEntrada, lista<cadenaCaracteres> info);

/*
* Comprueba si esta operación es válida para este nodo.
*
* @post No cambia el nodo, simplemente realiza las comprobaciones de esquemas necesarias para que se pueda realizar la operación que se propone. Se aplica el siguiente principio: "solo se pueden hacer aquellas operaciones que no modifiquen el esquema de salida resultante del nodo raiz".
*
* @ret true-> si la operación verifica ese principio.
* @ret false-> si la operación no cumple ese principio.
*/
logico isValidOperation(OperacionAlgebraica operacion)

/*
* Comprueba si el atributo de la condición se encuentra en el esquema pasado como argumento.
*
* @pre esquema.size()>0
*
* @post no modifica el objeto implicito
*
* @ret true-> si el atributo está en el esquema
* @ret false -> si el atributo no está en el esquema
*/
logico find(cadenaCaracteres condicion, vector<cadenaCaracteres>
```

```
    esquema) const;
```

Listing 3: Operaciones privadas *NodeLogico*

3.2.3 Operaciones públicas

```
public:
/*
* Constructor por defecto
*
* Crea un nodo completamente vacío y disfuncional.
*
* @post: this no es un nodo funcional.
*/
NodeLogico();


/*
* Constructor con parámetros
*
* Se encarga de construir un NodeLogico según la operación que se
    pasa como argumento.
*
* @param operacion: operación que alberga el nodo.
* @param esquemaEntrada: según la operación que sea se toma de una
    manera u otra.
*        TABLA, SELECCION, PROYECCION: solo se toma el primer
    esquema.
*        NATURAL JOIN, JOIN, PRODCART: se toman los dos primeros
* @param info: tiene una interpretacion u otra segun el valor de
    operacion:
*        TABLA: toma la primera cadena de caracteres como nombre de
    la tabla.
*        SELECCION: toma la primera cadena de caracteres como
    predicado de la seleccion.
*        PROYECCION: toma la lista entera como lista de atributos
    sobre los que proyectar.
*        NATURAL JOIN o JOIN: toma la primera cadena de caracteres
    como condicion de reunion.
*        PRODCART: no lo tiene en cuenta.
* @param children: vector de punteros de hijos del nodo, si no se
    proporciona toma un valor vacío y se interpreta como nodo hoja,
    es decir, debe ser TABLA.
* @param father: puntero al padre, si no se proporciona toma un
    valor nulo y se consideraria que el nodo es el raiz.
*
* @pre 0<esquemaEntrada.size()<=2
* @pre 0<=info.size()
*
* @post se crea el elemento OperacionAlgebraica asociado
* @post queda un nodo completamente funcional
*/
NodeLogico(TipoOperador operacion, pointer<vector<vector<
    cadenaCaracteres>>> esquemasEntrada,lista<cadenaCaracteres> info
    ,  vector<pointer<NodeLogico>> children={}, pointer<NodeLogico>
    father=nullptr);
```

```
/*
* Crea un nodo lógico identico al pasado como argumento
*
* @post other no se modifica ni se destruye.
*/
NodeLogico(pointer<NodeLogico> other);

/*
* Destructor de nodo
*
* Si tiene hijos, lo smanda eliminar, cuando esto termina entonces se eliminan las referencias y la memeoria dinámica que use. También se borra el enlace al padre si tiene.
*/
~NodeLogico();

/*
* Cambia el padre al padre que se pasa como argumento
*
* @param newFather: puntero al nodo padre.
*
* @post this.father==newFather
*/
void SetFather(pointer<NodeLogico> newFather);

/*
* Añade un nuevo hijo al nodo
*
* @param newSon: punter al nuevo nodo hijo
*
* @pre this.children->size() < 2
* @pre this.operation.operacion!=TABLE
*
* @post this.children->size()+=1
*
* @note este método no comprueba si la operación es válida, si se usa en optimización debe tenerse esto en cuenta.
*/
void AddSon(pointer<NodeLogico> newSon);

/*
* Impone la operacion que se pasa como argumento
*
* @pre si el nodo es hoja, solo se puede cambiar por una operacion TABLE
* @pre isValidOperation(operation)==true
*/
void SetOperacion(OperacionAlgebraica operation);

/*
* Impone el hijo i-esimo del nodo, no compueba que el cambio que se realice sea correcto y válido dentro del nodo con las estructuras de entrada
*
* @param i: posicion del hijo a sustituir.
```

```
* @param newSon: puntero al nuevo nodo hijo.
*
* @pre 0 <= i < this.children->size()
* @pre isValidOperation(newSon.operacion)==true
* @pre newSon debe ser del mismo tipo (hoja, interior) que this.
    children[i]
*
* @post this.children[i] == newSon
* @post newSon.father == this
*
*/
void SetSon(entero i, pointer<NodeLogico> newSon)

/*
* Devuelve la operacion del nodo
*
* @ret this.operacion
*/
OperacionAlgebraica getOperation() const;

/*
* Devuelve un puntero al nodo padre.
*
* @ret this.father
*/
pointer<NodeLogico> getFather() const;

/*
* Devuelve el hijo i-esimo del nodo
*
* @pre 0<=i<this.children->size()
*
* @ret this.children[i]
*/
pointer<NodeLogico> getSon(entero i) const;

/*
* Devuelve el numero de hijos
*
* @ret this.children->size()
*/
entero getNumSons() const;

/*
* Devuelve el esquema de salida de la operacion.
*
* @ret this.operacion.esquemaSalida
*/
vector<cadenaCaracteres> getOutputSchema() const;

/*
* Devuelve el esquema de entrada de la operacion.
*
* @ret this.operacion.esquemaEntrada
*/
vector<vector<cadenaCaracteres>> getInputSchema() const;
```

```
/*
* Realiza el intercambio de este nodo, solo cuando sea necesario. Este metodo actua de una manera u otra dependiendo de las operaciones en juego.
* Casos segun la operacion del hijo:
*       Si es SELECCION: simplemente se realiza el intercambio, pero el nodo actual recibe en su esquema de salida el atributo o los atributos de seleccion.
*       Si es PROYECCION: no se realiza un intercambio, se juntan los nodos uniendo los esquemas de ambas proyecciones, el puntero del hijo del nodo actual pasa a ser el puntero del hijo del siguiente nodo.
*       Si es NATURAL JOIN o JOIN: no se realiza un intercambio, se mantiene la proyeccion actual y se desplazan los hijos del join cambiandolos por dos nuevos nodos cuyos esquemas de salida son la interseccion del esquema de salida de la proyeccion junto con el esquema de entrada del join uniendo el atributo de reunion. Los antiguos hijos del join son ahora hijos de las proyecciones.
*       Si es PRODCART: se realiza el intercambio de la siguiente manera: Los hijos del producto cartesiano pasan a ser la proyeccion cada uno, cuyos hijos son las antiguas entradas del producto cartesiano.
*       Si es TABLA: no se realiza el intercambio.
*
* @pre Los esquemas resultantes deben ser coherentes, es decir, se realiza la comprobacion isValidOperation de forma constante.
* @pre this.operation.operacion == PROYECCION
*
* @post modifica el nodo actual, el puntero queda apuntando al primer nodo hijo por el que se intercambia.
*
* @ret true-> si se ha podido hacer la operacion.
* @ret false-> si no se puede hacer la operacion. (isValidOperation())
*/

logico InterchangeProjectionNode();

/*
* Devuelve si el nodo es raiz
*
* @ret true -> cuando father = nullptr and this.operacion.operation != TABLA
* @ret false -> cuando father != nullptr or this.operacion.operation == TABLA
*/
logico isRoot() const;

/*
* Devuelve si es un nodo hoja o no
*
* @ret true -> cuando children->size() == 0 and this.operacion.operation == TABLA
* @ret false -> cuando children->size() > 0 or this.operacion.operation != TABLA
```

```
*/
logico isLeaf() const;

/*
* Elimina la referencia al hijo i-esimo del vector de hijos.
*
* @param i posicion del hijo a eliminar.
* @pre 0 <= i < this.children->size()
*
* @post this.children->size()-=1
* @post Solo desvincula, no destruye la memoria del hijo.
*/
void RemoveSon(entero i);

/*
* Crea un nodo de reunión entre est hijo y su padre si es posible.
*
* @pre this.father->getOperation().operacion == SELECCION
* @pre operation.operacion == PRODCART
* @pre find(father->getOperation().condicion, operation.
    esquemasEntrada[0]) == true
* @pre find(father->getOperation().condicion, operation.
    esquemasEntrada[1]) == true
*
* @post no se modifica el objeto implícito
* @post el nodo resultante tiene como padre al abuelo del objeto
    implicito y como hijos a los hijos del objeto implicito. Además,
     como esquema de salida tiene el esquema de salida de su padre y
     como esquemas de entrada los del objeto implicito.
*
* @ret Nodo nuevo que es una fusión del producto cartesiano y la
    seleccion
*/
NodeLogico createJoin() const;
```

Listing 4: Operaciones públicas *NodeLogico*

3.3 TDA PlanLogico

Una vez se ha desarrollado este tipo de dato abstracto, el plan lógico queda reducido a muy poco; de hecho, podría no realizarse este tipo de dato abstracto puesto que, dado un nodo, podemos considerar que éste es raíz de un árbol.

No obstante, con motivo de escenificar la creación de un plan lógico de una consulta, creamos este tipo de dato abstracto como un puntero a un nodo lógico que será la raíz y que tendrá los métodos de optimización de consultas.

3.3.1 Estructura

```
class PlanLogico{
    private:
        pointer<NodeLogico> root;
    /*Métodos*/
```

```
}
```

Listing 5: Estructura *PlanLogico*

3.3.2 Operaciones privadas

Listing 6: Operaciones privadas *PlanLogico*

3.3.3 Operaciones públicas

```
public:

/*
* Constructor por defecto
*
* Crea un árbol cuyo nodo raíz es vacío y disfuncional.
*/
PlanLogico();

/*
* Constructor basado en una consultaSql
*
* Crea un plan lógico que representa la consulta anterior. Comienza
    creando los nodos hoja y prosigue creando el arbol de abajo a
    arriba.
*/
PlanLogico(cadenaCaracteres consultaSql);

/*
* Constructor basado en un nodo
*
* Crea un plan lógico a partir de un nodo en adelante.
* @post this.root=root
*/
PlanLogico(pointer<NodeLogico> root);

/*
* Destructor de PlanLogico
*
* Solo manda destruir la raiz y se desencadena el borrado en
    cascada
*/
~PlanLogico();

/*
* Clona el plan lógico
*
* @ret plan lógico distinto al objeto implícito pero que tiene el
    mismo contenido
*/
PlanLogico clone() const;

/*
* Devuelve la raiz del arbol para poder manejarlo
```

```
*
* @post el objeto queda modificable y es el usuario quien debe usarlo correctamente.
*/
pointer<NodeLogico> getRoot();

/*
* Devuelve un plan lógico donde se ha aplicado una vez la heurística de proyecciones
*
* @param nodo: nodo logico del arbol al que aplicarlo
*
* @ret nodo hijo del pasado como argumento, por tanto, como tenemos el puntero padre, podemos recuperar el PlanLogico.
*
*/
NodeLogico applyProjectionHeuristic(pointer<NodeLogico> nodo);

/*
* Devuelve un plan lógico donde se ha aplicado una vez la heurística de producto cartesiano.
*
* @param nodo: nodo lógico del árbol al que aplicarlo.
*
* @ret nodo lógico hijo del pasado como argumento, por tanto, como tenemos el puntero padre podemos recuperar el plan lógico.
*/
NodeLogico applyProdCartHeuritic(pointer<NodeLogico> nodo);
```

Listing 7: Operaciones públicas *PlanLogico*

Una vez se ha visto esto, podemos pensar algunos aspectos:

- **¿Para qué sirve la función *clone()*?**. Su utilidad puede ser mucho mayor de la que voy a comentar pero, cuando queremos crear un nuevo plan optimizado, no he encontrado una forma donde se pueda crear dicho plan con las modificaciones directamente; entonces, mi solución ha sido clonar el plan y después aplicar el algoritmo, para así generar las soluciones de forma recursiva.
- **¿Por qué usar *getRoot()*?**. La causa es lo anterior, es decir, para poder aplicar el algoritmo, que requiere un nodo, necesito recuperar la raiz de alguna manera. Además, la forma de movernos por el árbol será haciendo uso de los nodos. Por tanto, como el tipo de dato abstracto *TDA NodeLogico* se ha creado de forma lo más robusta posible, creo que no es demasiado problema; eso sí, si el usuario quiere o no tiene cuidado, puede romper el árbol sobre el que trabaja.
- **Si hay un *getRoot()*...¿Por qué no un *setRoot()***. Incluir esta operación, considero que sería un gran error puesto que podría romperse el esquema resultante de la consulta (incluso el propio resultado completo); entonces, creo que es mejor que los cambios de raíz, si son necesarios, se hagan de forma controlada.

4 TDA PlanFisico

Ahora, debemos plantear cuál sería la estructura de nuestro *TDA PlanFisico*; siguiendo la lógica que hemos trabajado con el *TDA PlanLogico*, construiremos el *TDA NodeFisico* y, con él, construiremos el plan físico.

4.1 TDA NodeFisico

Tal y como vimos en clase, el plan físico no es más que un plan lógico que contiene, en cada nodo, el coste de la operación y cómo se va a realizar esa operación [4]. Por simplicidad, supondremos que cada operación sólo tiene una forma de resolverse; por tanto, cada nodo del plan físico podemos entenderlo como un nodo lógico que tiene el coste de la operación.

De hecho, para poder consultar el coste del plan de forma sencilla, vamos a incluir el coste acumulado del nodo, es decir, el coste de realizar la consulta hasta ese nodo. Esto es una ventaja a la hora de consultarlo, pues solo deberemos hacer *root.costeAcumulado* para saber el coste del plan. Como desventaja, la creación del plan físico asociado a un plan lógico es un poco más costosa.

En vista de lo comentado, he decidido implementar el nodo físico como un tipo de nodo lógico que es más específico; de manera que, podrá realizarse lo mismo que en un nodo lógico [5] aunque esto no sea necesario pues su utilidad en el plan físico será muy reducida.

4.1.1 Estructura

```
class NodeFisico hereda NodeLogico{
    private:
        entero costePropio;
        entero costeAcumulado;
        vector<vector<enteros>> tamEsquemasEntrada;

    /*Métodos*/
}
```

Listing 8: Estructura *NodeLogico*

Como podemos ver, no sólo contiene el coste, sino que también contiene el tamaño de los atributos de los esquemas de entrada a la operación, esto es una pequeña mejora para no tener que consultar continuamente el catálogo y facilitar el cálculo del coste.

Además, podría surgir una duda: **¿Por qué los costes son enteros?** Considero que tratar datos con decimales no es algo relevante para estimar, y recalco **estimar**, el coste de un plan físico. Como vimos en teoría, no buscamos el plan más óptimo

[4] Realmente, una misma operación puede realizarse de varias maneras.

[5] Realmente, en un plan físico no se cambian los nodos de lugar ni se modifica el plan como tal, sólo se realiza el cálculo de costes.

pues esto puede llevar un tiempo elevado; entonces, podemos considerar el coste como un dato entero [6].

4.1.2 Operaciones privadas

```

private:
/*
* Calcula el coste propio de este nodo. Esta cantidad depende de la operacion que se realiza. Siempre se calculan el numero de operaciones de entrada/salida que se realizan; por tanto, se supone que se conoce el tamaño del bloque.
*
* @pre nodo: nodo lógico al que calcular el coste.
* @ret coste propio del nodo.
*/
entero ownCost(pointer<NodeLogico> nodo)const;
}
```

Listing 9: Operaciones privadas *NodeFisico*

4.1.3 Operaciones públicas

```

public:

/*
* Crea un nodo físico a través de un nodo lógico.
*
* @param nodo: nodo lógico que da la estructura del nodo físico
* @param tamEsquemasEntrada: tamaños de los esquemas de entrada, se toman tantos como necesite nodo.operation.operacion (la lógica es similar a la de info)
*
* @pre nodo != nullptr
*
* @post el nodo físico que se ha creado contiene el contenido del nodo lógico pero calcula su coste propio y acumulado.
*/
NodeFisico( pointer<NodeLogico> nodo, vector<vector<enteros>> tamEsquemasEntrada);

/*
* Devuelve el coste acumulado del nodo físico
*
* @ret this.costeAcumulado
*/
entero getAcumCost() const;

/*
* Devuelve el coste del nodo físico
*
```

[6] De hecho, en los ejercicios lo tratábamos así, tomando la parte entera o la función techo según el caso.

```
* @ret this.costePropio
*/
entero getOwnCost() const;
```

Listing 10: Estructura *NodeLogico*

El único método público más destacable es la creación de un nodo físico a través de un nodo lógico, pues será el método más útil en la creación de un plan físico mediante un plan lófico. Como podemos ver, requiere el tamaño de los esquemas de entrada asociado al nodo, para poder calcular costes de forma más sencilla.

4.2 TDA PlanFisico

Una vez hecho esto, un plan físico solo necesita el nodo físico raíz para existir, por lo que esta será su estructura [7]. Además, como funcionalidades principales del plan físico, debemos poder crearlo a través de un plan lógico dado y debemos poder conseguir su coste.

4.2.1 Estructura

```
class PlanFisico{
    private:
        pointer<NodeFisico> root;
    /*Métodos*/
}
```

Listing 11: Estructura *PlanFisico*

4.2.2 Operaciones privadas

En este caso, no he encontrado operaciones privadas necesarias. Si estuviéramos en el caso de controlar cómo se ejecuta una operación, podría haber un algoritmo de comparación de estos casos.

4.2.3 Operaciones públicas

```
public:

/*
* Constructor por defecto.
*
* Crea un árbol cuyo nodo raíz es vacío, disfuncional y sin coste.
*/
PlanFisico();

/*
* Constructor basado en un plan Lógico.
*
* Crea un plan lógico a partir de un nodoLógico en adelante.
*
```

[7] Si nos fijamos, se está siguiendo la misma lógica que en el *TDA PlanLogico*.

```
* El procedimiento se basa en crear el plan fisico desde los nodos hoja hacia el nodo que se pasa como argumento.
*
* @param root: nodo lógico que hace de raíz en el árbol.
* @param tamEsquemasEntrada: tamaños de los esquemas de las relaciones involucradas.
* @post this.root contiene el nodo raiz del plan fisico por lo que para obtener el costo solo hay que hacer this.root.getAcumCost()
*/
PlanFisico(pointer<NodeLogico> root, vector<vector<enteros>> tamEsquemasEntrada);

/*
* Destructor de PlanFisico.
*
* Solo manda destruir la raiz y se desencadena el borrado en cascada.
*/
~PlanFisico();

/*
* Devuelve el costo global del plan lógico.
*
* @ret coste del plan físico.
*/
entero getCost() const;
```

Listing 12: Operaciones públicas *PlanFisico*

5 Heurística proyeciones

Vamos a realizar el algoritmo que aplica la heurística sobre proyecciones en el plan lógico, es decir, sin tener en cuenta los costes ni variabilidades de los atributos. Sólo nos fijaremos en los atributos de esquemas que tienen los nodos y realizaremos los cambios necesarios.

Tenemos que tener en cuenta ciertas premisas:

- **El objetivo**. Normalmente, nuestro objetivo será bajar todo lo posible las proyecciones; el motivo de esto es que cuanto más rápido se eliminen los atributos que no intervienen en la consulta, más fluido irá el proceso de cálculo y menos costoso será el plan físico asociado al plan lógico.
- **Se debe respetar el esquema resultante**. Debemos respetar el esquema resultante de la consulta; además, ciertas operaciones como las selecciones o las reuniones requieren de ciertos atributos de entrada; por tanto, debemos mantener esos atributos hasta que se realice dicha operación.

En el código de abajo, se representa el algoritmo que se seguiría para implementar *applyProjectionHeuristic(nodo)*. Cabe destacar que su implementación recae muy fuertemente en la operación pública del *TDA NodeLogico* encargada de intercambiar dos nodos, ya que es la encargada de verificar que se pueda intercambiar o no, así

como de realizar el intercambio [8].

```
NodeLogico applyProjectionHeuristic(pointer<NodeLogico> nodo):
    // Condicion de parada 1: (Base)
    si noda->isLeaf() entonces
        retornar nodo;

    // Aplicamos el algoritmo de forma recursiva
    Para i desde 0 hasta nodo->getNumSons()-1 hacer:
        nodo->setSon(i,applyProjectionHeuristic(nodo->getSon(i))

    // Solo lo aplicamos si el nodo es proyeccion

    si nodo->getOperation().operacion == PROYECCION entonces:
        si nodo->isRoot() entonces:
            nodo.setSon(0,nodo);
            retornar nodo.getSon(0);

        // Si no es raiz intercambiamos los hijos respetando los
    esquemas -> de eso se encarga interchangeProjectionNode();

        nodo->InterchangeProjectionNode();
        retornar nodo.getSon(0);
```

Listing 13: Heurística proyecciones

Ahora bien, como podemos ver, este algoritmo modifica el plan lógico actual, perdiéndolo. Este no es el objetivo, debemos crear uno nuevo que será la copia del actual, por lo que bastaría usar la función *clone()* y después aplicar este algoritmo al plan lógico inicial sobre la raíz del mismo obtenida con *getRoot()* para conseguir el cambio local que buscamos. Repitiendo este proceso tanto como queramos podríamos obtener todos los planes lógicos necesarios.

6 Heurística del producto cartesiano

Esta heurística se basa en, dado un nodo que es un producto cartesiano, ser capaz de juntarlo con una selección que se encuentra justo encima. El nodo resultante es un nodo de tipo JOIN o NATURAL JOIN.

En nuestro *TDA PlanLogico* de esta heurística se encarga el método *applyProdCartHeuristic(nodo)*; donde la lógica de aplicación será la siguiente:

- Si el nodo es de tipo producto cartesiano y el nodo padre es una selección, vamos a intentar juntarlos en una reunión. Solo podemos juntarlos si es una igualdad y ambos esquemas de entrada del producto cartesiano tienen los atributos de igualdad y son del mismo tipo.
- Si no es producto cartesiano no hacemos nada y devolvemos el nodo actual.

[8] Las casuísticas de intercambio se encuentran en la especificación de este método y utiliza los métodos *getOutputSchema* y *getInputSchema* que servirán para las comprobaciones de los esquemas para determinar si la operación se puede realizar.

```
NodeLogico applyProdCartHeuristic(pointer<NodeLogico> nodo):

    //Condicion de parada 1: (Base)
    si nodo->isLeaf() entonces:
        retornar nodo;

    //Aplicamos el algoritmo de foma recursiva
    Para i desde 0 hasta nodo->getNumSons()-1 hacer:
        nodo->setSon(i,applyProdCartHeuristic(nodo->getSon(i)));

    //Solo lo aplicamos si el nodo es producto cartesiano

    si nodo->getOperation().operacion == PRODCART and !nodo->isRoot
   () entonces:
        pointer<NodeLogico> father = nodo->getFather();

        si father->getOperation().operacion == SELECCION and
        nodo->find(father->getOperation().condicion, nodo->
   getOperation().esquemasEntrada[0]) and
        nodo->find(father->getOperation().condicion, nodo->
   getOperation().esquemasEntrada[1]) entonces:

            //Creamos el nodo de join
            NodeLogico join = nodo.createJoin();

            si !join.empty() entonces:
                retornar join;

    retornar nodo;
```

Listing 14: Heurística producto cartesiano

Al igual que pasaba con la heurística sobre proyecciones, el algoritmo realiza modificaciones sobre el plan lógico sobre el que se ejecuta. Entonces, debemos primero clonar nuestro plan lógico inicial para obtener una copia de seguridad y después aplicarlo sobre la raíz.